

```
// Adding some rings to collect
rings = game.add.group();

// We will also enable physics for any ring that is
// created in this group
rings.enableBody = true;

// Here we'll create 10 of them evenly spaced apart
for (let i = 0; i < 10; i++)
{
    // Create a ring inside of the 'rings' group
    let ring = rings.create(i * 70, 0, 'ring');

    // Add some gravity
    ring.body.gravity.y = 300;
}

// The pointsText displays current score in top right
// corner of canvas
pointsText = game.add.text(624, 16, 'Points: 0', {
    fontSize: '32px', fill: '#000' });

// Using keyboard controls
cursors = game.input.keyboard.createCursorKeys();
}

function update() {

    // Collide the player and the rings with the platforms
    game.physics.arcade.collide(player, platforms);
    game.physics.arcade.collide(rings, platforms);

    // Checks if the player overlaps with any of the rings,
    // if he does call the collectRing function
    game.physics.arcade.overlap(player, rings, collectRing,
    null, this);

    // Reset the players velocity (movement), if no key is
    // pressed
    player.body.velocity.x = 0;

    // If left or right key is pressed
    if (cursors.left.isDown)
    {
        // Move left
        player.body.velocity.x = -150;
    }
    else if (cursors.right.isDown)
    {
        // Move right
        player.body.velocity.x = 150;
    }
    else
    {

```

```
// Dont move
player.animations.stop();
player.frame = 4;
}

// Allows the players to jump only if they are touching
// the ground.
if (cursors.up.isDown && player.body.touching.down)
{
    player.body.velocity.y = -350;
}
}

function collectRing (player, ring) {

    // Removes the ring from the screen, when player touches
    // the ring
    ring.kill();

    // Add and update the score points
    points = points + 10;
    pointsText.text = 'Points: ' + points;
}

}
```

Here, on the first line, we have created an instance of a *Phaser.Game* object and assigned it to the game variable. The first two parameters in the *Phaser.Game* function are the width and height of the canvas that Phaser creates. The third parameter can be *Phaser.CANVAS*, *Phaser.WEBGL* or *Phaser.AUTO*, which are for rendering the setting you want to use for your game. I recommend that you use *Phaser.AUTO*, which will automatically try to use either WebGL or Canvas, based on whichever the browser supports. The fourth parameter is an empty string. Here, you can give the ID of a DOM element where you would like to add the canvas element on the page. We have left it blank, so by default it will be appended to the body element. The last parameter is an object that consists of three references to Phaser's essential functions.

The essential functions are *preload()*, *create()* and *update()*. **preload():** This is used to load the assets needed for our game. PhaserJS will automatically look for this function when it starts, and load any asset defined within it.

create(): This function is used to create the game scene. Here, we add the assets loaded such as the background image, platform, player/characters and other game sprites.

update(): This function is called by the core game loop in every frame. It helps to check what is happening in every frame, like if there is a collision between the player and object, score updates, etc.

Finally, you should end up with the following directory structure:

```
assets/
• platform.png
```